ENGINEERING ARCHITECTURE PORTFOLIO

Authoritative realtime play, designed end to end.

System architecture, security boundaries, realtime coordination, persistence strategy, database integrity, and deliberate scaling decisions.

A technical case study of a multiplayer drawing-and-guessing platform built with Next.js, React, Go, WebSockets, and PostgreSQL.

Realtime core

Actor-style room loops serialize concurrent gameplay commands.

Security posture

HttpOnly sessions and 30-second, single-use WebSocket tickets.

Data integrity

Transactions, composite keys, triggers, checks, and partial unique indexes.

01 / EXECUTIVE SUMMARY

The architecture in one minute

Mithril Tiles separates fast-changing match state from durable business data while keeping the Go server authoritative over every competitive decision.

3	12	30
RUNTIME LAYERS	CORE DATA ENTITIES	TEST FILES

The central invariant is simple: **browsers request actions; the room server decides outcomes.** Drawer selection, word secrecy, guess validation, scoring, timers, and lifecycle transitions never depend on client honesty.

Next.js product edge

Renders the experience, validates browser inputs, proxies authenticated HTTP calls, and stores backend bearer credentials in secure HttpOnly cookies.

Go authority layer

Owns identity checks, authorization, origin enforcement, rate limits, room coordination, game rules, and the realtime protocol.

PostgreSQL durability layer

Preserves identities, catalog content, secure credentials, bot profiles, match history, rounds, awards, and final rankings.

Decision: transient and durable state are intentionally split.

Connections, timers, secret words, strokes, and chat live in memory; completed outcomes and identity data live in PostgreSQL.

Decision: each room is a serialized command processor.

Typed Go channels funnel joins, guesses, strokes, lifecycle events, and bot completions through one event loop.

Decision: the browser never receives a reusable backend token.

The frontend exchanges its HttpOnly session for a room-bound ticket that expires after 30 seconds and is deleted atomically when used.

Decision: database rules defend the model independently.

Application validation is reinforced with constraints, transactions, composite foreign keys, triggers, and partial uniqueness.

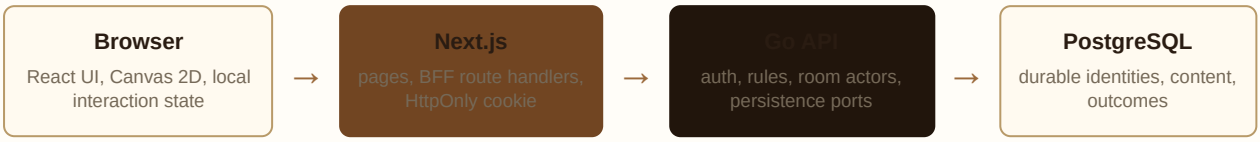
TECHNOLOGY PROFILE

Next.js 16 · React 19 · TypeScript · TanStack Query · Zustand · Zod · Go · WebSockets · PostgreSQL · pgx · Canvas 2D · Vitest

02 / SYSTEM LANDSCAPE

Two paths, one authority

Short-lived business operations use authenticated HTTP. Latency-sensitive room traffic uses a direct WebSocket after a secure ticket exchange.



HTTP control path

- register, sign in, and guest identity
- session restoration and logout
- word-pack and bot-profile catalog access
- room setup, bot membership, and game start
- final-score and participant lookup

WebSocket data path

- join, leave, and presence snapshots
- authorized drawing strokes and canvas revisions
- guess submission and scoring outcomes
- private drawer word vs. public masked word
- timers, round transitions, and final results

Why a Backend-for-Frontend?

Next.js route handlers adapt browser requests to the Go API and attach bearer tokens server-side. This reduces credential exposure, centralizes browser-facing validation, and gives the UI a stable same-origin API surface.

Why a direct socket?

Once authenticated, the browser connects directly to the Go room server. The long-lived, bidirectional channel avoids proxying every stroke through the rendering tier while retaining strict room authorization.

Why one authoritative service?

Gameplay decisions and match persistence sit behind one Go boundary. This keeps rules consistent across humans and bots and prevents clients from racing independent versions of game state.

> **Boundary rule:** Next.js owns presentation and credential containment. Go owns domain truth. PostgreSQL owns durable invariants.

The API composes panic recovery, trusted-origin CORS, bearer authentication, role checks, endpoint-specific rate limiting, and request deadlines. The HTTP server also defines read, write, and idle timeouts.

03 / TRUST BOUNDARY

Credential containment without sacrificing realtime latency

The login token stays outside browser JavaScript. A disposable, narrowly scoped credential is used for the socket upgrade.

WebSocket ticket sequence

1. **Authenticate over HTTP.** Next.js stores the backend bearer token in an HttpOnly, secure session cookie.
2. **Request room access.** A same-origin route handler reads the cookie server-side and asks Go for a ticket.
3. **Issue a scoped secret.** Go generates 32 cryptographically random bytes, hashes the plaintext with SHA-256, binds it to one principal and room, and gives it a 30-second TTL.
4. **Upgrade directly.** The browser presents only the ticket to the room WebSocket endpoint.
5. **Consume atomically.** PostgreSQL deletes and returns the matching unexpired row in a transaction. Replay attempts fail because the row no longer exists.
6. **Rehydrate identity.** Go reloads the active user or guest principal before accepting room participation.

Threats addressed

Threat	Control
Token theft via browser script	HttpOnly session cookie
Ticket replay	delete-on-consume transaction
Cross-room reuse	room-code binding
Stale credentials	30-second expiry
Database disclosure	only SHA-256 hash stored
Cross-origin socket abuse	trusted Origin patterns

Unified principal model

Registered users and temporary guests enter domain code through one principal abstraction. Routes can require any authenticated principal, a registered user, or an administrator without duplicating identity logic.

Authorization remains contextual

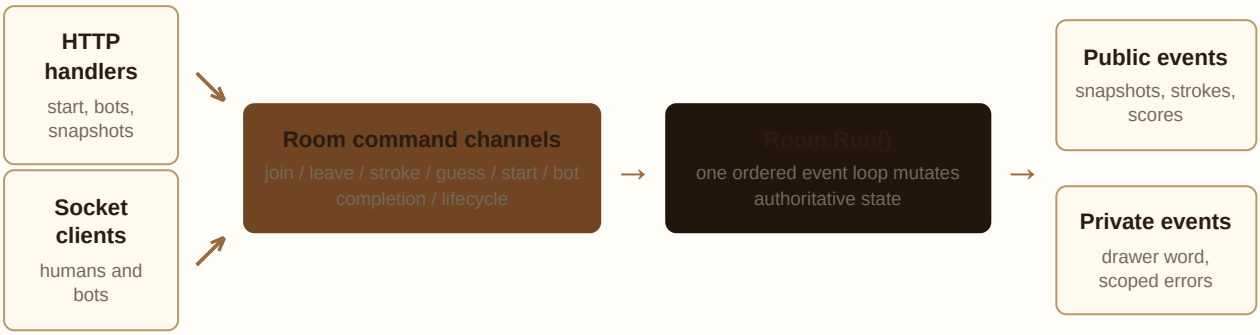
Authentication proves identity; room commands still verify role and state. Only the host can start a game or change bots, and only the current drawer can publish strokes.

> This design deliberately treats WebSocket establishment as a privileged security transition—not merely a transport handshake.

04 / CONCURRENCY MODEL

A room is an actor, not a shared bag of state

Every active room owns its match state and processes typed commands through a single select loop, turning concurrent network input into an ordered stream of decisions.



State owned by each room

- connected players, bots, host, and stable principal IDs
- game and round state, current drawer, private word
- scores, correct-guess tracking, and drawer rotation
- canvas revision and current-round stroke flow
- bounded chat ring buffer and reconnect sessions
- round cancellation, timers, and bot runtimes

Bounded pressure points

Stroke, guess, bot-completion, and room-information channels use explicit buffers. Chat history uses a fixed-capacity ring buffer rather than unbounded growth.

Race reduction by ownership

The event loop, rather than arbitrary socket goroutines, performs lifecycle mutations. Mutexes remain around the few state regions accessed outside the loop.

Domain commands carry identity

Guess and host actions include participant or principal IDs plus game/round context, so stale or forged commands can be rejected.

Slow work reports back as events

Persistence and bot providers can execute outside the main loop, then send typed completion messages back for ordered application.

Lifecycle cancellation is explicit

Round-scoped contexts stop timers and bot work when a round changes; room closure is protected by a one-time close guard.

Engineering payoff: the concurrency model aligns with the domain boundary. One room is one consistency scope.

Explicit states make illegal transitions visible

The room broadcasts consequences, not authority. Clients render snapshots and events but do not select the drawer, reveal the secret, award points, or advance the game.

Lobby host + roster → **Start transaction** game + participants → **Active round** draw + guess + timer → **Round commit** scores + completion → **Finalization** rankings + close

Information partitioning

Drawer: receives the private word and permission to emit normalized strokes.

Guessers: receive a masked word, public strokes, timer updates, and score events.

Everyone: receives membership snapshots, lifecycle state, chat history, and final standings.

Protocol defensiveness

- structured events are parsed against typed schemas in the frontend
- unsupported or malformed messages fail into explicit error paths
- canvas revisions distinguish current state from stale drawing events
- normalized coordinates make strokes independent of viewport size

Action	Server-side gate
Join room	principal identity, room capacity/state
Start game	host ID, player count, idle state, unique room code
Add/remove bot	host ID, active profile, idle state, uniqueness
Draw stroke	current drawer, active round, matching game context
Submit guess	participant identity, active game and round, not drawer
Advance round	timer/correct-guess outcome inside room lifecycle
Finalize game	authoritative score snapshot and persistence result

Late join and recovery

Room snapshots reconstruct roster, game state, canvas revision, and scoring context. A bounded message history supplies recent chat without turning the room into an event archive.

> The protocol is designed around **least information**: private knowledge is sent only to the participant who needs it.

06 / PERSISTENCE STRATEGY

Persist business outcomes, not transport noise

The live room keeps ephemeral mechanics close to the game loop. PostgreSQL records the facts required for identity, administration, auditing, and completed-match history.

In-memory, by design

- socket connections and reconnect activity
- current drawer and secret word
- round timers and cancellation functions
- live strokes and canvas revision
- recent chat ring buffer
- temporary correct-guess tracking
- bot runtime scheduling and perceptions

Reason: these values are high-frequency, round-scoped, latency-sensitive, or intentionally disposable.

Durable, by design

- registered users, guest identities, and hashed tokens
- one-time WebSocket ticket hashes
- word packs, words, and bot profiles
- game settings snapshot and lifecycle status
- participant identity snapshots
- round metadata and score awards
- final score and rank for each participant

Reason: these values carry product history, security meaning, content governance, or relational integrity.

Transactional game lifecycle

Boundary	Atomic work	Failure behavior
Start game	choose word; create game; snapshot all participants; assign host and drawer; create first round	rollback leaves no partial match
Start round	resolve/new participants; choose a word from the game pack; create sequential round	room does not publish a false start
End round	complete active round; resolve principals; write score awards	round and awards commit together
End game	sort standings; persist final scores; mark game complete	retry policy distinguishes transient from non-retryable errors

Snapshot fields protect history. Display names, selected word text, and settings are copied into match records so later profile or catalog edits do not rewrite the past.

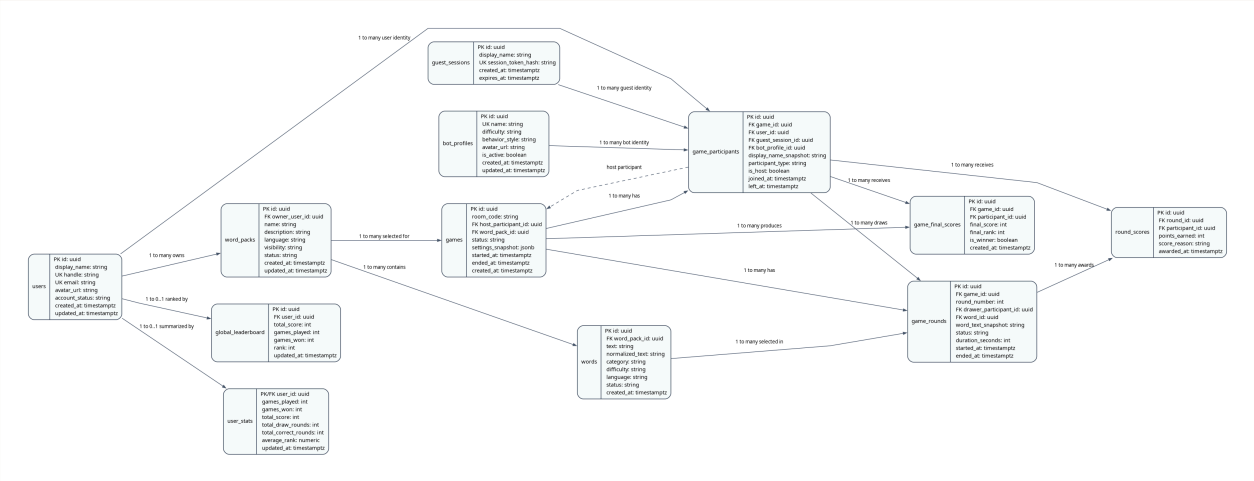
Deliberate non-goals

Raw replay streams, incorrect-guess timelines, full chat archives, and provider prompts are not stored. Adding them later should be a retention and analytics decision—not an accidental side effect of gameplay.

07 / DATABASE ARCHITECTURE

Integrity is encoded where the data lives

Twelve related entities model identity, secure access, content, participants, rounds, awards, and final outcomes.



Polymorphic identities, constrained

Users, guests, and bots share participant records, but a check constraint requires exactly the identity column matching the participant type.

Triggers derive denormalized integrity columns

Round word-pack and score game IDs are populated from their parents, then protected with composite constraints.

Composite ownership keys

Foreign keys prove that drawers, score recipients, and final-score participants belong to the same game—not merely that each UUID exists.

Deferred host relationship solves insertion order

The game and its host participant are created in one transaction; a deferred foreign key validates the cycle at commit.

Partial uniqueness models active state

Only one started game per room, one started round per game, one host per game, and one outstanding ticket per principal-room pair.

Domain values are checked, not implied

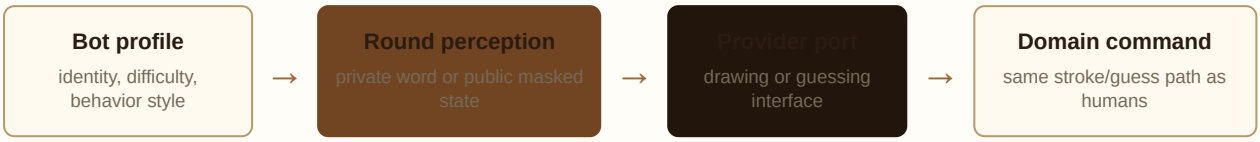
Lifecycle states, roles, difficulties, score reasons, positive durations, nonblank names, and chronological timestamps are database rules.

Deletion policy	Rationale
Game → participants, rounds, awards, finals: cascade	A match aggregate is removed as one unit.
Identity → tokens and socket tickets: cascade	Credentials cannot outlive their owner.
Word pack → words: cascade; referenced game history: protect	Catalog cleanup is allowed until durable history depends on it.

08 / BOT ARCHITECTURE

AI participants use the same rules as humans

Bots are first-class principals backed by persistent profiles, while provider interfaces keep game mechanics independent of any model vendor.



Provider strategy

- **Groq** when its API key is configured
- **xAI Grok** as the next configured option
- **Gemini** as another supported adapter
- **deterministic guessing** and **template drawing** as offline fallbacks

Behavior policy

Difficulty and style become round-scoped controls: guess delay, attempt budget, minimum revealed letters, drawing delay, and maximum stroke count.

No privileged scoring path

A generated guess becomes a normal typed guess command. The room validates it and awards points exactly as it would for a human.

No privileged drawing path

Generated strokes still pass current-drawer authorization and use the normalized drawing protocol.

Cancellation follows the round

Each runtime receives a scoped context, so late provider work cannot leak into a later round.

Completion returns to the room loop

Asynchronous provider output is applied only through an ordered bot-completion event.

> The key design choice is **ports before providers**. Model vendors are replaceable infrastructure; drawing and guessing remain domain capabilities.

Extension path

Future capability	Where it plugs in	What stays unchanged
new model vendor	provider adapter	room rules, scoring, protocol
new personality mode	bot profile + behavior policy	identity and participant schema
provider observability	adapter metrics/tracing	game lifecycle

09 / FRONTEND ARCHITECTURE

A feature-oriented client around a strict protocol edge

The UI is not a second game engine. It validates payloads, renders authoritative events, and isolates drawing, authentication, room state, and administration into focused modules.

App routes

Landing, login, registration, guest flow, room entry, live room, rules, about, and protected admin surfaces.

Feature modules

Authentication, drawing geometry, realtime protocol, room lifecycle, bot profiles, and word-pack management.

State tools

TanStack Query for server/session state, Zustand for room state, React Hook Form and Zod for validated inputs.

Realtime client lifecycle

```
idle → requesting_ticket →
connecting_socket → connected →
reconnecting / failed
```

Connection states are explicit UI states. Ticket rejection, malformed protocol events, and socket failures surface as controlled errors rather than silent desynchronization.

Responsive canvas math

Pointer coordinates are normalized into a device-independent range, clamped, transmitted, and denormalized for each viewer. A stroke therefore means the same thing on different canvas sizes.

Runtime validation at boundaries

Zod schemas validate authentication envelopes, room codes, ticket responses, admin forms, and structured socket events.

Server-only credential access

Browser calls target same-origin Next.js handlers; only server code reads the session cookie and forwards bearer authorization.

Snapshot plus events

Snapshots provide a recovery baseline; incremental events keep interaction responsive. Canvas revisions prevent stale drawing state from being trusted.

Shared error normalization

Backend validation and operational failures become a consistent browser-facing error shape.

Frontend responsibility: preserve interaction quality and render truth. **Backend responsibility:** create that truth.

Tested seams

geometry normalization · canvas emission · protocol parsing · ticket schemas · room reducer/state · socket failures · authentication forms · admin behavior

10 / RELIABILITY AND EVOLUTION

Known constraints are part of the design

The current architecture optimizes for strong single-process room consistency. Its scaling boundary is documented rather than hidden.

Defensive engineering already present

- HTTP read, write, and idle timeouts
- request-scoped database deadlines
- panic recovery and structured server errors
- trusted proxy-aware IP rate limiting
- trusted-origin CORS and socket origin checks
- transaction rollback on incomplete lifecycle work
- bounded chat and buffered realtime channels
- end-game retry classification
- room and round cancellation

Verification profile

15 Go test files cover room lifecycle, drawing authorization, guesses, message history, rate limiting, WebSocket tickets, persistence, and end-to-end API behavior.

15 frontend test files cover forms, schemas, drawing geometry, protocol parsing, room state, room entry, and shell behavior.

Current scale boundary

Active rooms live in one Go process. Horizontal replicas would hold independent room maps and could split participants unless routing guarantees room affinity.

Therefore the safe current deployment is one authoritative realtime process.

Phase 1: operational hardening

Add room/channel metrics, structured tracing, connection gauges, lifecycle latency, provider latency, and persistence retry counters.

Phase 2: room-affine scale-out

Partition rooms by code with sticky routing or a gateway that consistently selects the owning node.

Phase 3: distributed ownership

Introduce leases or a registry, a pub/sub backplane, and recoverable room snapshots only when scale justifies the consistency cost.

Phase 4: event products

Add replay, analytics, or leaderboards through explicit event retention and aggregation—not by overloading transactional tables.

> A good architecture communicates both **why it works now** and **what must change next**. Mithril Tiles makes that tradeoff explicit.

11 / REPOSITORY EVIDENCE MAP

Every major claim is traceable to code

Use this map during a technical interview or repository review to move directly from architecture decision to implementation.

Concern	Primary evidence	What to inspect

System composition	`cmd/api/main.go`
	`cmd/api/routes.go`
	`cmd/api/server.go`
		dependency wiring, middleware composition, timeouts, endpoint boundaries

Room actor model	`internal/realtime/room.go` `internal/realtime/room_manager.go`	typed channels, single select loop, room ownership, bounded buffers
Game rules	`internal/realtime/handlers.go` `internal/realtime/game_start.go`	host/drawer authorization, lifecycle transitions, scoring, private/public events
Secure socket entry	`cmd/api/websocket.go` `internal/data/websocket_tickets.go`	30-second issue, SHA-256 storage, room binding, atomic consume, origin checks
Transactional persistence	`cmd/api/game_lifecycle.go` `internal/realtime/gamelifecycle.go`	domain port, start/round/end transactions, retry semantics
Database invariants	`migrations/000004...000008`	checks, partial indexes, composite foreign keys, triggers, role model
Bot abstraction	`internal/realtime/guess_provider.go` `internal/realtime/drawing_planner.go`	provider interfaces, adapters, deterministic fallbacks
Credential boundary	`frontend/src/lib/auth/session-cookie.ts` `frontend/src/app/api/`	HttpOnly storage, same-origin BFF routes, server-side bearer forwarding
Realtime client	`frontend/src/features/realtime/` `frontend/src/features/rooms/room-shell.tsx`	protocol schemas, ticket lifecycle, connection states, snapshot/event rendering
Canvas portability	`frontend/src/features/drawing/`	normalization, clamping, pointer handling, deterministic tests

Architectural strengths

- authority and trust boundaries are explicit
- concurrency model matches the domain
- persistence is transactional and intentionally scoped
- database invariants prevent cross-aggregate corruption
- AI integrations are behind stable domain ports
- limitations and evolution paths are documented

Interview discussion prompts

- Why tickets are safer than exposing bearer tokens
- When actor-style state beats database-backed coordination
- How composite constraints protect game ownership
- Why replay data was intentionally excluded
- How to scale without weakening per-room ordering

> **Design thesis:** keep realtime decisions close, durable facts correct, credentials narrow, and boundaries easy to explain.

Prepared from the repository implementation and versioned migrations. Supporting documents: `ARCHITECTURE.md` and `database_design.md`.